[C]1

AINS HACKATHON

Technical Specification

AI for Enterprise Automation

Engineering the next layer of intelligent enterprise systems

Organised by **AINS 4.0** in partnership with **Vectors**

Contents

1 Global Requirements

1.1 Context & Mission

The central mission is to build AI systems that address documented, structural gaps in enterprise automation – gaps that exist today in production environments, at scale, where conventional software has already reached its limits. All three use cases share the same foundational constraint: *remove the AI component, and the system ceases to function entirely.* The intelligence is not a layer bolted on top of existing tools; it is the mechanism that makes the system possible.

1.2 Technical Freedom

There is no imposed technology stack beyond the Atlassian ecosystem constraint. Teams are free to choose any combination of frameworks, models, databases, and orchestration tools that best serve their use case.

1.3 What a Good Solution Must Achieve

1. **AI is the mechanism, not a feature.** The system cannot function without the AI component. If removing the AI leaves a working (if degraded) tool, the solution does not meet the requirement.

2. **Actionable outputs.** The system produces structured, traceable outputs – verdicts, audit trails, briefings, scored assessments, grounded recommendations – not raw inference or open-ended generation.

3. **Structural intelligence beyond retrieval.** The system reasons over live enterprise signals to produce classification, detection, attribution, or generation that conventional automation cannot perform. Keyword search and template filling are not sufficient.

4. **Explainability.** Every output is traceable. Judges and users can understand why the system produced a given result, what evidence it used, and where it is uncertain.

5. **Evaluation.** The team defines at least one measurable evaluation metric and runs it on a test set, however small. Non-deterministic systems must address how they handle evaluation.

1.4 What Is Not Allowed

- Standalone chatbot or conversational assistant projects – a chat layer may exist as a secondary interface, but the core system must perform non-trivial reasoning, classification, or generation beyond answering questions.

- Solutions where the AI is a wrapper around an existing Atlassian feature – teams must address a gap that the current Atlassian stack demonstrably does not fill.

- Solutions that reproduce a commercially available product without meaningful original contribution to the problem space defined in this brief.

- Prototypes that require manual setup, hardcoded values, or human intervention to produce their core output during the demo.

> The goal is not to prescribe what you build – it is to ensure you build something that could not exist without genuine AI engineering. The use case defines the problem space and the acceptance criteria. Design, architecture, product decisions, and technology choices are entirely yours.

1.5 Submission Requirements

1.5.1 First Submission – Concept and Prototype Foundation

The first submission is an early-stage deliverable demonstrating that the team has understood the problem, committed to a technical direction, and begun building.

- **Concept presentation** – problem framing, proposed solution, target users, core AI mechanism, and value proposition; format is free (slide deck, document, or mockup); maximum 10 pages or slides
- **GitHub / GitLab repository link** – must contain at minimum: a structured README describing the chosen use case and approach, initial project structure, and any early code, mockups, or architectural sketches, even if not yet functional

> **First submission is not a finished prototype.** Judges will evaluate clarity of problem understanding, technical direction credibility, and early evidence of execution – not completeness. A well-structured empty repository with a strong README is more valuable than a working but misdirected prototype.

1.5.2 Final Submission

Every team must submit the following on demo day:

- **Pitch deck** – problem, solution, target users, value proposition, demo walkthrough, limitations, and next steps (maximum 15 slides)
- **Demo video** – a recorded walkthrough of at least one end-to-end scenario demonstrating a core capability (maximum 5 minutes)
- **Architecture diagram** – system components, data flow, AI pipeline
- **Data description** – sources used, formats, key fields, quality notes, and sensitivity handling
- **Explainability layer** – a mechanism or interface element showing why the system produced a specific output, including evidence, confidence, and decision trace where applicable
- **Evaluation report** – metric(s) used, test protocol, and results on a test set
- **GitHub / GitLab repository** – source code with a clear README and setup instructions

1.6 Non-Functional Expectations

- **Responsiveness.** Core outputs should be produced within a reasonable time for a realistic synthetic dataset; latency choices must be justified.
- **Reliability.** The system handles missing, ambiguous, or inconsistent enterprise data without crashing; document how edge cases are handled.
- **Scalability mindset.** Explain how the system would behave with ten times the data volume, concurrent processes, or a broader enterprise deployment.

2 Use Case 1 – Continuous Evaluation System for Non-Deterministic AI Agents

An AI System that Continuously Evaluates the Behaviour of Enterprise AI Agents Across Full Execution Trajectories and Produces Auditable, Actionable Verdicts

2.1 Problem

Enterprise teams are deploying AI agents that do not behave deterministically. The same instruction, given twice, can produce different tool calls, different reasoning chains, and different outputs. Traditional unit tests – which pass or fail based on exact output matching – are structurally incompatible with this class of system.

The consequence in production is severe. An agent that looks busy, reasons intelligently, and calls the right-looking tools can still fail to complete the task – and the failure only becomes visible after a real action has been taken: a Jira ticket incorrectly modified, a Confluence page overwritten, a JSM workflow incorrectly routed. By the time failure is detected, the damage is done.

Inside the Atlassian ecosystem, this gap is directly documented. When an agent configuration or underlying model changes, there is no mechanism today to detect whether the agent's behaviour has drifted from its intended specification. What does not exist is a system that captures the full execution trajectory of an agent run, evaluates it at multiple levels, attributes any failure to a specific component, and does this continuously – not as a one-shot pre-deployment check, but as an ongoing discipline applied to every production run.

> **What this use case is not asking teams to build:** Teams are not asked to design a specific evaluation methodology or define what "correct" agent behaviour looks like in a particular domain. Those are design decisions for the team. The problem is the infrastructure gap: the absence of a general-purpose system capable of evaluating non-deterministic agent trajectories in a production Atlassian environment.

2.2 Illustrative Scenarios

The following examples illustrate the kind of situations this system should address. They are not prescriptive – teams are free to define their own scenarios, agent types, and evaluation targets.

- **Scenario A – Silent failure:** An agent triages incoming JSM tickets, assigning them to the correct team and setting the priority field. On a given day, 12 out of 80 tickets are assigned incorrectly – but no alert fires because the agent completed its tool calls without an error. A continuous evaluation system captures the full trajectory of each run, compares the assignment decision against a behavioural specification, and flags the 12 divergent cases with an explanation of where the classification went wrong.
- **Scenario B – Drift after model update:** An agent that has been stable for three months begins producing longer, less structured Confluence summaries after an underlying model update. No functionality broke, but output quality has measurably shifted. A continuous evaluation system detects this drift by comparing output characteristics across runs, surfaces a drift alert
- **Scenario C – Component-level failure attribution:** A multi-step agent fails to complete a task. The end-to-end verdict is "failure," but the useful signal is *which step* failed: did retrieval return irrelevant context? Did planning produce a broken tool call sequence? Did execution call the right tool with wrong parameters? A continuous evaluation system

attributes the failure to the specific component, enabling the team to fix the right thing.

2.3 Core Capabilities Expected

- **Trajectory capture** – record the full execution trace of an agent run: every reasoning step, every tool call with its parameters and return value, every output produced
- **Multi-level evaluation** – assess the trajectory at at least two levels (e.g. end-to-end task completion and component-level tool call correctness); evaluation must not rely solely on exact output matching
- **Failure attribution** – when evaluation fails, identify which component in the trajectory caused or contributed to the failure
- **Drift detection** – compare evaluation results across runs over time and detect meaningful shifts in agent behaviour, tool usage patterns, or output characteristics
- **Human-readable verdict** – produce an output a non-AI-specialist engineer can act on: what the agent was supposed to do, what it actually did, where it diverged, and what the recommended action is

2.4 Acceptance Criteria

Criterion	What to Demonstrate	Priority
Trajectory capture works	Full execution trace of at least one agent run is captured and inspectable	Must
Multi-level evaluation	At least two evaluation levels are implemented and produce distinct signals	Must
Failure attribution	At least one demonstrated case where failure is attributed to a specific component, not just flagged at the end-to-end level	Must
Human-readable verdict	Every evaluation run produces a structured, actionable report	Must
Drift detection	At least one demonstrated case of drift detected across two or more runs	Should
Non-determinism addressed	Team documents how they handle the fact that re-running the same evaluation may produce different results	Should
Evaluation of the evaluator	Team defines and reports at least one metric assessing the quality of the evaluation system itself	Should

2.5 Key Considerations

- **The evaluator uses AI to evaluate AI.** An LLM-as-judge pattern, a behavioural contract comparison, or a simulation-based approach are all valid directions. What matters is that the evaluation methodology is principled and documented.
- **Non-determinism is the central technical challenge.** A system that only works when the agent behaves identically every time has not addressed the problem. Teams must design evaluation logic that produces meaningful signals even when the agent takes different paths to the same (or different) outcome.

- **Scope discipline.** Teams should define a specific agent type and task domain as their evaluation target. A system that claims to evaluate any agent at any task will likely evaluate no agent well.

3 Use Case 2 — Agent Execution Tracer and Deterministic Replay Engine

An Observability and Debugging Infrastructure Layer That Captures the Exact State, Tool Calls, and Trajectories of AI Agents to Enable Deterministic Replay

3.1 Problem

When a traditional software system fails, you look at the stack trace, reproduce the bug, and fix it. When an agentic AI fails in production – perhaps it chose the wrong tool, hallucinated an argument, or got stuck in a reasoning loop – a simple log file is inadequate. You cannot easily reproduce the bug because the environment state and the LLM's output change on every run.

If an engineer tries to debug a failed Jira triaging agent by running it again, the agent might make completely different decisions, or worse, execute a side-effecting tool (like sending a customer email) during the debug session.

What is missing is an enterprise-grade "Flight Recorder": an infrastructure layer that transparently captures every LLM input, context variable, and tool-call payload during a live run, with a Replay Mode that allows developers to re-execute the agent step-by-step using the exact recorded tool responses – guaranteeing a deterministic run for debugging without touching live APIs.

3.2 Illustrative Scenarios

- **Scenario A – The Side-Effect Trap:** An agent designed to audit Confluence pages and email page owners drafts an aggressive email. To debug the prompt, the engineer needs to replay the exact scenario without actually sending the email. The Flight Recorder mocks the email tool, allowing the engineer to tweak the system prompt and deterministically test the agent's behaviour.
- **Scenario B – Divergence Testing:** An agent fails to resolve a JSM ticket because it queried a database tool with incorrect SQL syntax. The engineer uses the Replay Harness to step through the trace, injects the correct SQL at the exact moment of failure, and lets the agent continue its trajectory to verify subsequent steps.
- **Scenario C – Auditing and Compliance:** A compliance officer needs to understand exactly what context an agent had access to when it approved a sensitive workflow three weeks ago. The Flight Recorder provides a perfectly preserved state snapshot of that exact run.

3.3 Core Capabilities Expected

- **Trajectory recording** – transparently intercept and log every LLM call, prompt, context variable, and tool-call response (payload, latency, status) during a live agent run
- **Deterministic replay** – re-execute the agent step-by-step, intercepting any tool calls and returning the recorded responses instead of hitting live endpoints
- **State snapshotting** – serialize and store the agent's memory/context window at each step so it can be resumed or inspected
- **Divergence support** – allow developers to modify a variable or prompt during replay and see how the agent's trajectory diverges from the original recorded path

3.4 Acceptance Criteria

Criterion	What to Demonstrate	Priority
Record functionality works	A live agent run is successfully recorded, capturing at least one LLM call and one tool call	Must
Deterministic replay	The recorded run is replayed successfully without triggering the live external tool	Must
State inspection	The user can inspect the exact context/prompt sent to the LLM at a specific step	Must
Divergence editing	A developer modifies a prompt or tool result during replay and the agent takes a new path	Should

3.5 Key Considerations

- **Proxying is the primary challenge.** Building the proxy that seamlessly intercepts tool calls (whether via REST, MCP, or native function calling) without altering the agent's core architecture is the primary technical hurdle.
- **Visualisation and usability.** A raw JSON dump of a trace is difficult to parse. Teams should think about how to present the execution graph clearly to a human engineer.

4 Use Case 3 – AI Automation for the Atlassian Workspace

A Freestyle Challenge to Design and Build a High-Value, AI-Driven Enterprise Workflow That Solves a Complex Problem Conventional Automation Cannot Handle

4.1 Problem

Many enterprise processes are currently stuck in an "uncanny valley" of automation: too repetitive for humans to enjoy doing, but requiring too much semantic reasoning for traditional RPA or standard webhooks to handle.

If an automation relies on simple "IF/THEN" logic (e.g., "IF ticket priority is HIGH, THEN alert Slack"), it does not need AI. However, if it requires reading a messy thread, understanding context, negotiating a schedule, or summarising technical code, traditional software fails.

Teams are challenged to identify a high-value, reasoning-intensive workflow within an enterprise context and build an AI agent (or multi-agent system) to automate it end-to-end within the Atlassian workspace.

4.2 Illustrative Scenarios

- **Scenario A – The Semantic Duplicate Resolver:** Standard Jira duplicate detection relies on exact keyword matching. A team builds an agent that reads incoming bug reports, checks them against the historical Jira backlog using semantic vector search, determines if the bug is a true duplicate despite different phrasing, and automatically links the tickets while writing a polite explanation to the reporter.
- **Scenario B – The ADR Generator:** A team builds an agent that listens to Slack or Jira comment threads where technical decisions are debated. Once a decision is reached, the agent drafts a formal Architecture Decision Record, categorises it, and publishes it to Confluence.
- **Scenario C – The Automated Code Reviewer & Planner:** An agent detects when a Jira story is moved to "In Progress," pulls the relevant repository context, and proactively drafts a technical implementation plan and potential security risks directly into the Jira ticket.

4.3 Core Capabilities Expected

- **Autonomous reasoning** – the system must make decisions based on unstructured input (text, code, or messy data) rather than rigid rule-sets
- **Atlassian integration** – the workflow must read from and/or write to Atlassian tools seamlessly via supported APIs or frameworks
- **Graceful degradation** – if the AI is uncertain, it must fail safely by looping in a human rather than executing a destructive or incorrect action
- **Structured output** – the final output must be formatted, actionable, and human-readable, rather than raw chat dialogue

4.4 Acceptance Criteria

Criterion	What to Demonstrate	Priority
End-to-end workflow works	The system completes the chosen business workflow from trigger to final output successfully	Must
AI necessity verified	The team must prove their workflow could not be built using standard Jira Automation or rigid IF/THEN rules	Must
Actionable output produced	The automation produces a structured artifact (a drafted document, a routed ticket, a scored report), not just an open-ended chat response	Must
Evaluation metric defined	The team defines what "success" looks like for their specific workflow and measures it on a test dataset	Should

4.5 Key Considerations

- **Beware the "thin wrapper."** A system that simply takes user input, passes it wholesale to an LLM, and pastes the raw result back into Jira will score poorly. The best projects will chain multiple prompts, utilise structured extraction, or orchestrate multiple tools to complete a robust task.
- **Scope discipline.** Choose a highly specific business problem. "An agent that writes marketing copy" is too broad. "An agent that converts closed JSM tickets into public-facing FAQ Confluence pages" is a well-scoped target.

5 Evaluation & Judging Framework

5.1 Judging Criteria

All projects will be evaluated by a panel of judges including AINS and Vectors representatives, technical AI engineering experts, and enterprise software practitioners. Scores are assigned across five dimensions:

Dimension	What Judges Look For	Weight
Engineering Depth	Quality of the core AI engineering; non-trivial problem solving at the infrastructure layer; evidence that the AI mechanism is the system, not a feature bolted on top of conventional software	50%
Prototype Quality	End-to-end demo works with realistic synthetic inputs; handles edge cases; produces structured, inspectable outputs without manual intervention during the demo	25%
Explainability & Auditability	Outputs are traceable; the system communicates uncertainty; a non-AI-specialist engineer can understand why the system produced a given result and what to do about it	15%
Evaluation & Rigour	Team defined and ran a real evaluation protocol with documented metrics, a synthetic test set, and reported results; non-determinism is addressed	10%

5.2 Bonus Points

- **Protocol gap addressed and documented** – the team identifies a specific, documented gap in tool protocols or evaluation architectures and implements a concrete solution, with clear documentation of the problem and the approach
- **Self-evaluation** – the system includes a mechanism to assess the quality of its own outputs and surface uncertainty or low-confidence results to human reviewers
- **Real enterprise validation** – at least one real enterprise engineer or AI platform owner tested the prototype and provided documented feedback
- **Open contribution** – the team documents a specific evaluation pattern, debugging architecture pattern, or testing methodology as a reusable artefact (specification document, open-source module, or structured design decision record) that the broader community could use